

Aether — Security & Compliance

Enterprise-Grade Security Architecture

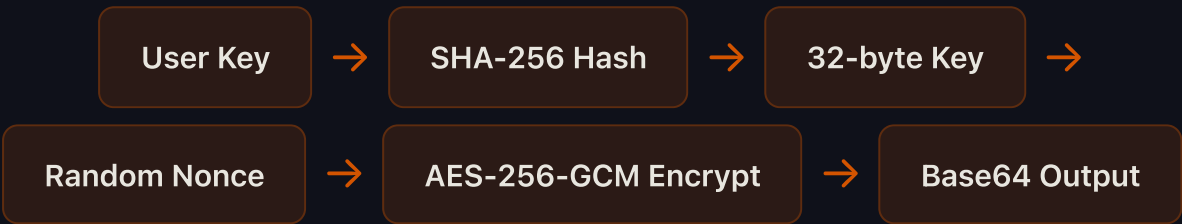
Enterprise security pack: RBAC, OIDC/SAML, encryption, audit integrity, isolation comparison — plus roadmap for SBOM and signed workloads.

Defense in Depth

Encryption at Rest

All sensitive data is encrypted using AES-256-GCM — the same authenticated encryption standard used by cloud providers and financial institutions.

Encryption Flow



How It Works

The user-provided encryption key is hashed with SHA-256 to produce a fixed 32-byte (256-bit) key. A cryptographically random 12-byte nonce is generated for each encryption operation. The AES-256-GCM cipher provides both confidentiality and authenticity — any tampering with the ciphertext is detected on decryption. The final output is base64-encoded for safe storage and transport.

Key Properties

Property	Value	Purpose
Algorithm	AES-256-GCM	Authenticated encryption with associated data
Key Derivation	SHA-256	Deterministic 32-byte key from any-length input
Nonce Size	12 bytes (96 bits)	Unique per encryption, cryptographically random

Auth Tag	16 bytes (128 bits)	Tamper detection and integrity verification
Output Format	Base64 (nonce + ciphertext)	Safe for JSON, YAML, and environment variables

```
# Encrypt a secret value
```

```
$ aether encrypt --key my-encryption-key --value "database-password-2024"
```

```
Encrypted: k8Jd2f...base64...Qx9a==
```

```
# Decrypt a value
```

```
$ aether decrypt --key my-encryption-key --value "k8Jd2f...base64...Qx9a=="
```

```
Decrypted: database-password-2024
```

API Authentication

Every API request is authenticated via Bearer token. CORS protection ensures only authorized origins can access the Aether control plane.

Authentication Flow



Middleware Stack



AETHER_API_KEY

Set via environment variable. Every request must include

`Authorization: Bearer <key>.`

Requests without a valid token receive 401 Unauthorized.



CORS Protection

Cross-Origin Resource Sharing headers restrict which domains can call the API. Prevents unauthorized browser-based access to the control plane.



Request Logging

Every authenticated request is logged with timestamp, actor, method, and path. Failed authentication attempts are flagged in the audit trail.



Rate Limiting

Built-in protection against brute-force attacks. Excessive failed authentication attempts trigger temporary lockout.

Configuration

```
# Set the API key via environment variable
$ export ETHER_API_KEY="your-secure-api-key-here"

# Start the Aether server with authentication enabled
$ aether serve --port 5090

# Authenticated API request
$ curl -H "Authorization: Bearer your-secure-api-key-here" \
    http://localhost:5090/api/workloads

# Unauthenticated request – rejected
$ curl http://localhost:5090/api/workloads
401 Unauthorized: Missing or invalid API key
```

*Security is not a feature you add later — it is a foundation you build on.
Every Aether API endpoint is authenticated by default with no opt-out.*

Audit Trail Integrity

Every action in Aether generates an immutable audit event with SHA-256 integrity hashes. Tamper detection is built into every log entry.

Audit Event Structure

```
{
  "timestamp": "2026-04-12T14:32:01.847Z",
  "actor": "admin@example.com",
  "action": "deploy",
  "target": "my-app",
  "runtime": "kubernetes",
  "result": "success",
  "details": { "replicas": 3, "image": "app:v2.1.0" },
  "integrity_hash": "sha256:alb2c3d4e5f6...64-char-hex"
}
```

Integrity Verification



SHA-256 Per Event

Each audit event includes a SHA-256 hash computed over its content. Any modification to the event data invalidates the hash.



Tamper Detection

When reading audit logs, Aether recomputes and verifies each integrity hash. Tampered entries are flagged with a warning.



Actor Tracking



Access Logging

Every event records the authenticated actor (API key identity) who performed the action. No anonymous operations.

Read access to sensitive resources is also logged, providing a complete picture of who accessed what and when.

Audit Commands

```
# View recent audit events
```

```
$ aether audit --last 20
```

```
# Filter by action type
```

```
$ aether audit --filter deploy,migrate,delete
```

```
# Verify audit trail integrity
```

```
$ aether audit --verify
```

```
Verifying 1,247 audit events...
```

```
All events verified: 0 tampered, 0 missing
```

```
# Export audit trail for compliance
```

```
$ aether audit --export json --output audit-report.json
```

Policy Engine

Enforce organizational rules on every deployment. The policy engine evaluates workloads against configurable rules before allowing operations.

Policy Rule Types

Rule	Production	Development	Description
Min Replicas	Enforced (≥ 2)	Disabled	Minimum replica count for high availability
Resource Limits	Required	Optional	CPU and memory limits must be specified
Health Checks	Required	Optional	Liveness and readiness probes mandatory
Image Registry	Allowlist only	Allowlist only	Images must come from approved registries
Privileged Mode	Blocked	Allowed	Privileged containers are not allowed in production
Root User	Blocked	Allowed	Containers must run as non-root in production
Network Policy	Required	Optional	Network segmentation policy must exist
Image Tag	No :latest	Any tag	Production images must use specific version tags
Secret Encryption	Required	Optional	All secrets must be encrypted at rest
Namespace Isolation	Enforced	Flexible	Workloads must deploy to designated namespaces

Policy Gate on Deploy

```
$ aether deploy my-app --runtime kubernetes --env production
```

```
Policy violation: 2 rules failed
```

```
[FAIL] min-replicas: spec has 1 replica, minimum is 2 for production
```

```
[FAIL] health-checks: no liveness probe defined
```

```
[PASS] resource-limits: CPU and memory limits set
```

```
[PASS] image-registry: image from approved registry
```

```
Deployment blocked. Fix violations or use --skip-policy to override.
```

```
# Override policy (requires explicit flag and audit logging)
```

```
$ aether deploy my-app --runtime kubernetes --env production --skip-policy
```

```
WARNING: Policy check skipped — this action is logged in the audit trail
```

State Security

Aether protects its internal state with atomic writes, advisory file locking, and restrictive file permissions — preventing corruption and unauthorized access.

Protection Layers



Atomic Writes

State is written to a temporary file first, then renamed to the target path. This ensures the state file is never partially written — even during power loss or crash.



Advisory File Locking

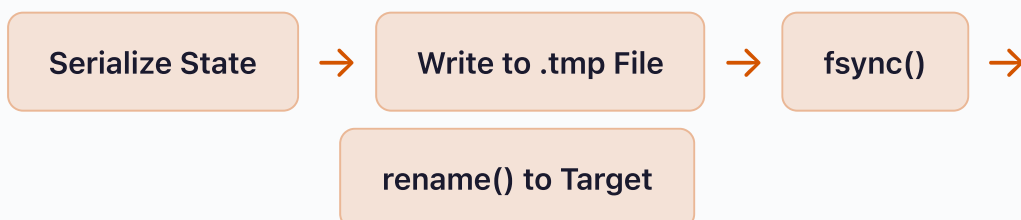
Uses POSIX `flock` to prevent concurrent state modifications. Multiple Aether processes coordinate safely through kernel-level advisory locks.



File Permissions

Backup and snapshot files are created with `00600` permissions (owner read/write only). No group or world access to sensitive state data.

Atomic Write Flow



Why Atomic Writes Matter

The POSIX `rename()` syscall is atomic on all major filesystems. By writing to a temporary file first and then renaming, Aether guarantees that the state file is

always in a consistent state. If the process crashes during the write, the temporary file is simply orphaned — the original state file remains intact.

File Permission Model

File Type	Permission	Octal	Access
State file	rw-----	0o600	Owner read/write only
Backup files	rw-----	0o600	Owner read/write only
Snapshot files	rw-----	0o600	Owner read/write only
Audit log	rw-----	0o600	Owner read/write only
Config directory	rwx-----	0o700	Owner full access only

Kubernetes Resource Safety

Production-grade Kubernetes interactions with timeouts, conflict resolution, cleanup guarantees, and strict input validation.



5-Minute Timeouts

All Kubernetes API operations have a 5-minute timeout. No operation can hang indefinitely — if the API is unresponsive, the operation fails cleanly with a descriptive error.



409 Conflict Handling

When a resource update conflicts (HTTP 409), Aether automatically retries with the latest resource version. Optimistic concurrency control prevents lost updates.



Resource Cleanup on Failure

If a deployment fails mid-way, Aether cleans up partially-created resources. No orphaned Deployments, Services, or ConfigMaps left behind.



Namespace Validation

Target namespaces are validated before deployment. If the namespace does not exist, Aether creates it with appropriate labels rather than failing silently.

DNS-1123 Input Validation

All resource names, namespace names, and labels are validated against the DNS-1123 standard before any Kubernetes API call is made:

- Must be lowercase alphanumeric characters or hyphens
- Must start with an alphanumeric character
- Must end with an alphanumeric character

- Maximum 63 characters for labels, 253 for hostnames

```
# Invalid name caught before API call
$ aether deploy My_App --runtime kubernetes

Validation error: 'My_App' is not a valid DNS-1123 name
  - Uppercase characters are not allowed
  - Underscores are not allowed (use hyphens instead)
Suggestion: use 'my-app' instead
```

Safety Guarantees Summary

Scenario	Protection	Behavior
API unresponsive	5-min timeout	Clean failure with error message
Concurrent update	409 retry	Re-fetch resource, retry with latest version
Partial deployment	Cleanup	Delete all resources created in failed attempt
Invalid namespace	Auto-create	Create namespace with labels before deploy
Invalid name	Pre-validation	Reject before any API call is made

Secure Your Infrastructure

Aether's security is built-in, not bolted on. Configure your security posture in minutes.

Production-Ready Security

AES-256-GCM encryption, authenticated APIs, tamper-evident audit trails, and policy enforcement — out of the box.

Security Configuration Commands

```
# Set up API authentication
$ export AETHER_API_KEY="$(openssl rand -hex 32)"

# Encrypt sensitive values
$ aether encrypt --key "$ENCRYPTION_KEY" --value "db-password"

# Configure policy enforcement
$ aether policy set --env production --min-replicas 2 \
  --require-health-checks --block-privileged

# Verify audit trail integrity
$ aether audit --verify

# View security-related audit events
$ aether audit --filter auth,policy,encrypt

# Export compliance report
$ aether compliance-report --format pdf --output report.pdf
```

Security Documentation

Comprehensive security architecture guide with threat model, encryption details, and compliance mapping.

Compliance Templates

Pre-built policy sets for SOC 2, HIPAA, and PCI-DSS compliance. Deploy with confidence.